C++ OOP Notes

Constructors, Destructors, this Pointer & Copy Semantics

1. Constructors

A constructor is a special member function that is automatically called when an object is created. It has the same name as the class and no return type. Constructors initialize data members — if not explicitly set, values default to garbage.

Key properties:

- Same name as the class
- No return type (not even void)
- Must be declared in the public section
- Called only once per object — at creation time

> *Note: If no constructor is defined, the C++ compiler auto-generates a default constructor with an empty body.*

1.1 Default Constructor

A constructor with no parameters. Initializes members with predefined default values.

```
class class_name {
    int data_member1;
    string data_member2;
public:
    class_name() {              // Default constructor
        data_member1 = 69;
        data_member2 = "Coding Ninjas";
    }
};
```

1.2 Parameterized Constructor

Accepts arguments to initialize data members with custom values at the time of object creation. Different objects can be initialized with different values.

```
class class_name {
    int data_member1;
    string data_member2;
public:
    class_name(int num, string str) {    // Parameterized constructor
        data_member1 = num;
        data_member2 = str;
    }
```

```
};
```

1.3 Copy Constructor

Creates a new object by copying values from an existing object of the same class. The parameter must be passed by reference using & (ampersand).

```
class class_name {
    int data_member1;
    string data_member2;
public:
    class_name(class_name &obj) {        // Copy constructor
        data_member1 = obj.data_member1;
        data_member2 = obj.data_member2;
    }
};
```

> *Note: If a copy constructor is not defined, C++ provides a default one that performs memberwise (shallow) copy.*

1.4 Complete Example — Smartphone Class

```
class smartphone {
    string model;
    int year_of_manufacture;
    bool _5g_supported;
public:
    // Default constructor
    smartphone() {
        model = "unknown";
        year_of_manufacture = 0;
        _5g_supported = false;
    }
    // Parameterized constructor
    smartphone(string model_string, int manufacture, bool _5g_) {
        model = model_string;
        year_of_manufacture = manufacture;
        _5g_supported = _5g_;
    }
    // Copy constructor
    smartphone(smartphone &obj) {
        model = obj.model;
        year_of_manufacture = obj.year_of_manufacture;
        _5g_supported = obj._5g_supported;
    }
};

int main() {
    smartphone unknown;                        // Default constructor
    smartphone iphone("iphone 11", 2019, false);  // Parameterized
    smartphone iphone_2(iphone);               // Copy constructor
}
```

2. Constructor Overloading

Constructor overloading means defining multiple constructors in the same class, each with a different number or type of parameters. The compiler calls the appropriate constructor based on the arguments passed during object creation.

Criteria for overloading:

- Different number of parameters, OR
- Different types of parameters

```
class smartphone {
    string model;
    int year_of_manufacture;
    bool _5g_supported;
public:
    // 0 parameters
    smartphone() {
        model = "unknown";  year_of_manufacture = 0;  _5g_supported = false;
    }
    // 2 parameters
    smartphone(string model_string, bool _5g_) {
        model = model_string;  _5g_supported = _5g_;
    }
    // 3 parameters
    smartphone(string model_string, int manufacture, bool _5g_) {
        model = model_string;
        year_of_manufacture = manufacture;
        _5g_supported = _5g_;
    }
};

int main() {
    smartphone unknown;                        // 0 params
    smartphone redmi("Note 7 Pro", false);     // 2 params
    smartphone iphone("iphone 11", 2019, false); // 3 params
}
```

3. Destructors

A destructor is the opposite of a constructor — it is called automatically when an object is destroyed. Its purpose is to free resources (memory, file handles, etc.) acquired during the object's lifetime.

3.1 Syntax

```
~class_name() {
    // cleanup code
}
```

Rules for destructors:

- Name begins with ~ followed by the class name
- No return type (not even void)
- No parameters allowed — cannot be overloaded
- Only one destructor per class
- Must be declared in the public section
- Address of destructor cannot be accessed
- Compiler generates a default destructor if none is defined

3.2 When is a Destructor Called?

- When an object goes out of scope
- When the program ends
- When a scope block { } containing a local variable ends
- When delete is called on a dynamically allocated object

3.3 Example

```
#include <iostream>
using namespace std;

class Guided_path {
public:
    Guided_path() {
        cout << "Constructor is called" << endl;
        cout << "Welcome to Guided Path" << endl;
    }
    ~Guided_path() {
        cout << "Happy Learning" << endl;
        cout << "Destructor is called" << endl;
    }
};

int main() {
    Guided_path obj;  // constructor called
}             // destructor called automatically

// Output:
// Constructor is called
// Welcome to Guided Path
// Happy Learning
// Destructor is called
```

4. The this Pointer

The this pointer holds the address of the current object — i.e., it points to the object that invoked the member function. It is implicitly available inside all non-static member functions.

Main uses of this:

- Refer to a current class instance variable (resolve name conflicts)
- Pass the current object as a parameter to another method
- Declare indexers

4.1 Example — Resolving Name Conflicts

```
#include <bits/stdc++.h>
using namespace std;

class mobile {
    string model;
    int year_of_manufacture;
public:
    void set_details(string model, int year_of_manufacture) {
        this->model = model;                        // 'this->model' = data member
        this->year_of_manufacture = year_of_manufacture; // 'model' alone = local
param
    }
    void print() {
        cout << this->model << endl;
        cout << this->year_of_manufacture << endl;
    }
};

int main() {
    mobile redmi;
    redmi.set_details("Note 7 Pro", 2019);
    redmi.print();
}

// Output:
// Note 7 Pro
// 2019
```

> *Note: When a local variable and a data member share the same name, this-> distinguishes the data member from the local variable.*

5. Shallow Copy vs Deep Copy

5.1 Shallow Copy

An object is created by copying all field values directly, including pointer values. Both the original and the copy point to the same memory address. Changes via one object affect the other — this is usually undesirable.

> *Note: The C++ compiler's implicit copy constructor and assignment operator perform shallow copy by default.*

```
class students {
    int age;
    char* names;
public:
    students(int age, char* names) {
        this->age = age;
        this->names = names;   // Shallow copy — both point to same memory!
    }
};
```

5.2 Deep Copy

A new block of memory is allocated and the contents of the original are copied into it. Changes to one object do not affect the other. You must explicitly define the copy constructor to perform a deep copy.

```
class student {
    int age;
    char* names;
public:
    student(int age, char* names) {
        this->age = age;
        this->names = new char[strlen(names) + 1]; // Allocate new memory
        strcpy(this->names, names);                // Copy data into new memory
    }
};
```

6. Quick Reference Summary

Concept	Description	Key Point
Default Constructor	No parameters, sets default values	Called when object is created with no args
Parameterized Constructor	Takes arguments to initialize members	Allows different values per object
Copy Constructor	Copies values from another object	Param must be passed by reference (&)
Constructor Overloading	Multiple constructors with different params	Compiler selects based on arguments passed
Destructor	Frees resources when object is destroyed	Prefixed with ~, no params, only one allowed
this Pointer	Points to current object	Use to resolve name conflicts
Shallow Copy	Copies pointer values (same memory)	Default compiler behavior — risky

Deep Copy	Allocates new memory and copies data	Must be explicitly defined